

Module 4: Advanced ICG & Runtime Environment

Short-Circuit Booleans, Backpatching, Activation Records & Parameter Passing

Module IV: Advanced ICG & Runtime Environment — Topics Covered

1. Short-Circuit vs. Numerical Boolean Evaluation
2. Control Flow Translation (if-else, while, for)
3. Backpatching: makelist, merge, backpatch
4. Procedure Calls & Return Mechanisms
5. Runtime Memory Organization (Text, Data, Heap, Stack)
6. Activation Records (Stack Frames) & Parameter Passing

1. Translation of Boolean Expressions (Short-Circuit Evaluation)

In programming languages like C and Java, boolean operators `&&` and `||` evaluate conditionally:

- In $B_1 || B_2$: If B_1 is true, B_2 is skipped entirely.
- In $B_1 \&\& B_2$: If B_1 is false, B_2 is skipped entirely.

2. Backpatching Technique

In a single-pass compiler, forward jump targets are not known when the jump instruction is emitted. **Backpatching** leaves target addresses blank and fills them once the target label is generated.

- `makelist(i)` : Creates a new list containing just the jump instruction index i .
- `merge(p1, p2)` : Concatenates two lists of jump instructions $p1$ and $p2$.
- `backpatch(p, i)` : Inserts label i as the target for each instruction on list p .

3. Runtime Storage Organization & Activation Records

Runtime Memory Layout

Code / Text Segment
Static / Global Data
Heap (grows downward ↓)
Stack (grows upward ↑)

Structure of an Activation Record (Stack Frame)

Actual Parameters (Arguments passed by caller)
Return Value & Return Address (Program Counter PC)
Control Link (Dynamic Link) -> Points to caller's frame
Access Link (Static Link) -> Points to lexical parent frame
Saved Machine Status (Registers, Status Flags)
Local Variables & Temporaries

FIGURE 4.1: MEMORY LAYOUT & ACTIVATION RECORD ARCHITECTURE

4. Parameter Passing Mechanisms

Mechanism	How Arguments are Passed	Effect of Changes on Actual Arguments
Call by Value	Evaluates r-value and passes a copy to local parameter.	Changes to formal parameter have zero effect on caller.
Call by Reference	Passes l-value (memory address / pointer) of actual argument.	Changes directly mutate caller's variable .
Call by Name	Textual macro substitution (evaluated lazily via Thunk).	Re-evaluated each time parameter is referenced.